Ateneo de Davao University

4-334 · Full-Stack Web Development · 1st Semester, July–November 2026

Lab Activity 9 – Barangay Bakuna Tracker: Blade, Tailwind & Vite

Week 9 topics: Blade directives, layouts (@yield) vs components, Vite asset pipeline, Tailwind in Laravel.
Submission deadline: _____ **Repository:** `LabActivity9-Lastname` (public)

Scenario

Nurse Marites Abellana runs the vaccination desk of the **Barangay Matina Crossing Health Center** in Davao City. Her Week 2-style dashboard is a single static HTML page with hand-written Flexbox/Grid CSS, and every new page means copy-pasting the whole header and footer. The City Health Office has asked her to move it into Laravel so it can later connect to a real database (Week 10). Your job is to rebuild the **presentation layer only** as Blade views styled with Tailwind through Vite. Schedule data comes from a small seeded SQLite table (see Database Seeding below).

Personalize it: the dashboard title must be "*Bakuna Tracker – Purok [your last name]*", the logged-in health worker shown in the nav must use your own first name, and the vaccine schedule must contain at least one entry dated on your birthday (any year in 2026). Your seeded data must include these exact vaccines: *BCG, Pentavalent, OPV, MMR, Anti-Flu*, plus one vaccine of your choice.

Setup: Installing Tailwind CSS in Laravel (v3 + PostCSS)

> **Important:** New Laravel 12 projects (current `laravel/laravel` skeleton) ship with **Tailwind v4** preinstalled: `tailwindcss ^4` and `@tailwindcss/vite` in `package.json`, a `tailwindcss()` plugin in `vite.config.js`, and `@import 'tailwindcss';` plus `@source` lines in `app.css`. This lab follows the **v3** setup from the slides, so do Step 3 to remove the v4 setup first.

Step 1 – Create the project

```
composer create-project laravel/laravel LabActivity9-Lastname
cd LabActivity9-Lastname
npm install
```

Step 2 – Install Tailwind v3, PostCSS and Autoprefixer

```
npm install -D tailwindcss@3 postcss autoprefixer
npx tailwindcss init -p
```

This creates `tailwind.config.js` and `postcss.config.js`.

Step 3 – Remove the default v4 setup (if present)

```
npm uninstall @tailwindcss/vite
```

In `vite.config.js`, delete the line `import tailwindcss from '@tailwindcss/vite';` and the `tailwindcss()`, entry inside `plugins`. Check that `package.json` now shows `"tailwindcss": "^3..."`.

Step 4 – Set the content paths in `tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    './resources/**/*.blade.php',
    './resources/**/*.js',
    './resources/**/*.vue',
  ],
  theme: { extend: {} },
  plugins: [],
}
```

Step 5 – Replace everything in `resources/css/app.css` (remove `@import 'tailwindcss'` and the `@source` lines)

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Step 6 – Check `vite.config.js`

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';

export default defineConfig({
    plugins: [
        laravel({
            input: ['resources/css/app.css', 'resources/js/app.js'],
            refresh: true,
        }),
    ],
});
```

Step 7 – Load assets in your layout head (`resources/views/layouts/app.blade.php`)

```
<head>
    <meta charset="utf-8">
    <title>@yield('title', 'Bakuna Tracker')</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
```

Step 8 – Run two terminals while developing

```
# Terminal 1
php artisan serve
# Terminal 2
npm run dev
```

Step 9 – Quick test. Put this in a view and open `http://127.0.0.1:8000`. You should see a blue box with white text:

```
<div class="bg-blue-500 text-white p-4 rounded">Tailwind works!</div>
```

Step 10 – Before submitting, run `npm run build` and make sure `public/build/manifest.json` exists.

Troubleshooting

- **Styles not applying:** make sure `npm run dev` is running and that `tailwind.config.js` `content` includes `./resources/**/*.blade.php`. Classes in files Tailwind doesn't scan are removed.
- "**Vite manifest not found" error:** run `npm run dev` (development) or `npm run build` (production).
- **Error about `@tailwindcss/vite` or `@import 'tailwindcss'`:** leftover v4 setup. Redo Steps 3 and 5.
- **Do not use the Tailwind CDN** or a hardcoded `/css/app.css` link. Requirement 3 checks for them.

Setup: Database Seeding (SQLite)

New Laravel 12 projects use SQLite by default. In `.env`:

```
DB_CONNECTION=sqlite
```

The file `database/database.sqlite` is normally created for you. If it is missing, create it with `touch database/database.sqlite` (on Windows, `type nul > database\database.sqlite`). *Using MySQL/XAMPP instead?* Set `DB_CONNECTION=mysql`, `DB_DATABASE`, `DB_USERNAME` and `DB_PASSWORD` in `.env` and create the database first.

Step 1 – Create the model and migration

```
php artisan make:model Vaccination -m
```

Step 2 – Define the columns in `database/migrations/xxxx_create_vaccinations_table.php`:

```
public function up(): void
{
    Schema::create('vaccinations', function (Blueprint $table) {
        $table->id();
        $table->string('patient_name');
```

```
        $table->string('purok');
        $table->string('vaccine');
        $table->string('dose');          // e.g. 1st dose, Booster
        $table->date('scheduled_at');
        $table->string('status');        // Scheduled, Done, Overdue
        $table->timestamps();
    });
}
```

In `app/Models/Vaccination.php` add:

```
protected $fillable = ['patient_name', 'purok', 'vaccine', 'dose', 'scheduled_at', 'status'];
protected $casts = ['scheduled_at' => 'date'];
```

Step 3 – Create the seeder

```
php artisan make:seeder VaccinationSeeder
```

Write `database/seeders/VaccinationSeeder.php`. You need 6+ rows covering BCG, Pentavalent, OPV, MMR, Anti-Flu and one vaccine you choose, plus **one row dated on your birthday in 2026**. Use your own resident names. Don't copy this example:

```
use App\Models\Vaccination;

public function run(): void
{
    $rows = [
        ['patient_name' => 'Baby Juan Dela Cruz', 'purok' => 'Purok 3', 'vaccine' => 'BCG',
         'dose' => '1st dose', 'scheduled_at' => '2026-11-03', 'status' => 'Scheduled'],
        ['patient_name' => 'Lola Pacing Reyes', 'purok' => 'Purok 1', 'vaccine' => 'Anti-Flu',
         'dose' => 'Annual', 'scheduled_at' => '2026-10-20', 'status' => 'Overdue'],
        // ... Pentavalent, OPV, MMR, your own vaccine, your birthday row
    ];

    foreach ($rows as $row) {
        Vaccination::create($row);
    }
}
```

Step 4 – Register it in `database/seeders/DatabaseSeeder.php`:

```
public function run(): void
{
    $this->call([
        VaccinationSeeder::class,
    ]);
}
```

Step 5 – Migrate and seed

```
php artisan migrate --seed
```

```
# reset everything and reseed later:
php artisan migrate:fresh --seed
```

Step 6 – Verify

```
php artisan tinker
>>> App\Models\Vaccination::count();   // should be 6 or more
```

Step 7 – Load the data in the controller

```
use App\Models\Vaccination;

public function index()
{
    $title = 'Bakuna Tracker — Purok Lastname';
    $schedules = Vaccination::orderBy('scheduled_at')->get();
    $stats = [
        ['label' => 'Total', 'value' => $schedules->count()],
        ['label' => 'Done', 'value' => $schedules->where('status', 'Done')->count()],
        // ... 2 more
    ];
    $alerts = $schedules->where('status', 'Overdue');
    return view('dashboard.index', compact('title', 'stats', 'schedules', 'alerts'));
}
```

Troubleshooting

- "**no such table: vaccinations":** you haven't migrated. Run `php artisan migrate` or `migrate:fresh --seed`.
- **Seeder didn't insert anything:** check that it is listed in `DatabaseSeeder`'s `$this->call()`, or run it directly with `php artisan db:seed --class=VaccinationSeeder`.
- "**Database file does not exist":** create `database/database.sqlite` (see above) and make sure `DB_CONNECTION=sqlite`.
- **MassAssignmentException:** add the columns to `$fillable` in the model.

Setup: Simple Login with a Seeded User

This is a small manual login so you can test `@auth`/`@guest`. Don't install Breeze or Jetstream. Full authentication comes in Week 12.

Step 1 – Seed your user. Laravel already includes the `users` migration. Update `database/seeders/DatabaseSeeder.php`:

```
use App\Models\User;
use Illuminate\Support\Facades\Hash;

public function run(): void
{
    User::create([
        'name'     => 'Juan',                       // YOUR first name
```

```
        'email'    => 'nurse.delacruz@bakuna.test', // nurse.[yourlastname]@bakuna.test
        'password' => Hash::make('bakuna2026'),
    ]);

    $this->call([
        VaccinationSeeder::class,
    ]);
}

php artisan migrate:fresh --seed
```

Your seeded credentials: email `nurse.[yourlastname]@bakuna.test`, password `bakuna2026`. **List them in your README.**

Step 2 – Create the controller

```
php artisan make:controller AuthController

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class AuthController extends Controller
{
    public function showLogin()
    {
        return view('auth.login');
    }

    public function login(Request $request)
    {
        $credentials = $request->validate([
            'email'    => ['required', 'email'],
            'password' => ['required'],
        ]);

        if (Auth::attempt($credentials)) {
            $request->session()->regenerate();
            return redirect()->intended('/dashboard');
        }

        return back()->withErrors([
            'email' => 'Invalid email or password.',
        ])->onlyInput('email');
    }

    public function logout(Request $request)
    {
        Auth::logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return redirect('/login');
    }
```

```
}
```

Step 3 – Routes (`routes/web.php`). Both pages are protected:

```
use App\Http\Controllers\AuthController;
use App\Http\Controllers\DashboardController;

Route::get('/login', [AuthController::class, 'showLogin'])->name('login')->middleware('guest');
Route::post('/login', [AuthController::class, 'login'])->middleware('guest');
Route::post('/logout', [AuthController::class, 'logout'])->name('logout')->middleware('auth');

Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index']);
    Route::get('/schedule', [DashboardController::class, 'schedule']);
});

Route::redirect('/', '/dashboard');
```

Step 4 – Login view (`resources/views/auth/login.blade.php`). Style it your own way with Tailwind:

```
@extends('layouts.app')
@section('title', 'Sign in')
@section('content')
<form method="POST" action="/login" class="max-w-sm mx-auto mt-10 space-y-4 p-6 bg-white rounded
shadow">
    @csrf
    <input type="email" name="email" value="{{ old('email') }}" placeholder="Email"
           class="w-full border rounded p-2">
    @error('email') <p class="text-red-600 text-sm">{{ $message }}</p> @enderror

    <input type="password" name="password" placeholder="Password" class="w-full border rounded
p-2">
    @error('password') <p class="text-red-600 text-sm">{{ $message }}</p> @enderror

    <button class="w-full bg-emerald-600 text-white rounded p-2">Sign in</button>
</form>
@endsection
```

Step 5 – Nav in the layout

```
@auth
    <span>Hi, {{ auth()->user()->name }}</span>
    <form method="POST" action="{{ route('logout') }}" class="inline">
        @csrf
        <button class="underline">Logout</button>
    </form>
@endauth
@guest
    <a href="{{ route('login') }}">Sign in</a>
@endguest
```

Step 6 – Test it. Log out, then open `/dashboard`. You should be sent to `/login`. Sign in with your

seeded credentials and you should land on the dashboard with “Hi, [name]” in the nav. Click Logout to go back to `/login`.

Troubleshooting

- **419 Page Expired:** the form is missing `@csrf`. Both the login and logout forms need it.
- **Login always fails:** the user wasn't seeded (run `migrate:fresh --seed` and check `User::count()` in tinker), the email or password is wrong, or the password was saved without `Hash::make()`.
- **Redirect loop or “Route [login] not defined”:** the login route must be named `login` and must **not** be inside the `auth` group.
- **Logout gives 405 Method Not Allowed:** logout must be a POST form, not a link.

Requirements

1. **Migration, seeder & controller → view.** A `vaccinations` migration and `VaccinationSeeder` (6+ rows, registered in `DatabaseSeeder`) so that `php artisan migrate:fresh --seed` works. `DashboardController` loads `$schedules = Vaccination::orderBy('scheduled_at')->get()` and returns `view('dashboard.index', compact(...))` (dot notation mapping to `resources/views/dashboard/index.blade.php`) with `$title`, `$stats` (4 counts) and `$alerts`.
2. **Master layout with @yield.** `resources/views/layouts/app.blade.php` holds the nav, footer and `@yield('content')`, plus `@yield('title', 'Bakuna Tracker')` in the <title>. At least **two** pages (`/dashboard` and `/schedule`) `@extends('layouts.app')` and fill `@section('content')`.
3. **Vite in the layout.** The layout head uses `@vite(['resources/css/app.css', 'resources/js/app.js'])`; `vite.config.js` lists both entry points. No hardcoded `/css/app.css` link and no Tailwind CDN.
4. **Tailwind via the Laravel pipeline.** Tailwind is installed with PostCSS; `resources/css/app.css` imports the base/components/utilities layers; `tailwind.config.js` `content` includes `./resources/**/*.blade.php`. `npm run build` must succeed and produce `public/build`.
5. **@if / @elseif / @else.** An alert banner shows the alert count when `$alerts` has items, a different message for a special case (e.g., a vaccine out of stock), and “All clear” otherwise. Use `@unless` at least once (e.g., hide the “Overdue” badge unless a row is overdue).
6. **@forelse with $loop.** The schedule table renders with `@forelse … @empty … @endforelse` (show “No schedules yet.” when empty) and uses `$loop` (e.g., `$loop->odd` zebra rows, `$loop->iteration` row numbers, or highlighting `$loop->first`).
7. **Login with a seeded user + @auth/@guest.** A seeded user (your first name, `nurse.[lastname]@bakuna.test`) can sign in on `/login` (`@csrf`, `old()`, `@error`). The nav shows “Hi, [name]” and a POST Logout button inside `@auth`, and a “Sign in” link inside `@guest`. `/dashboard` and `/schedule` use the `auth` middleware so guests are sent to `/login`, and logout works.
8. **Blade components with props/slots.** Create at least two components in `resources/views/components/`: `<x-stat-card :title=".." :value=".." />` (used in a loop for the 4 stats) and one slotted component such as `<x-panel title="..">…</x-panel>`

wrapping the schedule table or alerts.

9. **Responsive utility layout + safe output.** Stats use a Tailwind grid like `grid gap-4 md:grid-cols-4` (stacked on mobile), with at least one `sm:/md:/lg:` prefix elsewhere; no custom media-query CSS. All dynamic data uses `{{ }}`; no `{!! !!}` on resident data.
10. **Authenticity.** Code, commits and README must look like your own work: meaningful commit history (not a single dump), your personalized data, and a short README with your **seeded login credentials** and how to run it (`php artisan migrate:fresh --seed`, `composer install`, `npm install`, `npm run dev`, `php artisan serve`). Screenshots optional.

Deliverables

- Public GitHub repo `LabActivity9-Lastname` with the full Laravel project (no `vendor/` or `node_modules/`).
- Submit the repo link on Moodle before the deadline. Be ready to defend your work in lab.

Rubric (10 points)

#	Criterion	Pts
1	Migration + seeder work; controller loads seeded data into a dot-notation view	1
2	Master layout with @yield; 2+ pages using @extends/@section	1
3	@vite in layout; vite.config.js entries; no hardcoded CSS/CDN	1
4	Tailwind via PostCSS/Vite; correct content paths; build succeeds	1
5	@if/@elseif/@else alert banner + @unless	1
6	@forelse/@empty with $loop usage	1
7	Seeded-user login, @auth/@guest nav, protected routes, logout	1
8	Two Blade components (props + slot) used properly	1
9	Responsive Tailwind grid/prefixes; escaped {{ }} output	1
10	AI authenticity: 1 = looks student-written, 0.5 = mixed, 0 = strongly AI-looking	1
	Total	**10**

Notes (not scored): lab presence and a public repository are required for the work to be checked. Scores are rounded down to a whole number.